

Report: Predict Bike Sharing Demand with AutoGluon Solution

SHAHEER FARDAN

Initial Training

What did you realize when you tried to submit your predictions? What changes were needed to the output of the predictor to submit your results?

As stated in the project python notebook file, Kaggle competition website doesn't accept negative test predictions. So, we are required to set any negative prediction scores to 0. Next, output has to be in the format specified in sampleSubmission.csv.

What was the top ranked model that performed?

WeightedEnsemble_L3 was the top ranked model during 3rd phase where we did hyperparameter tuning along with addition of new features.

Exploratory data analysis and feature creation

What did the exploratory analysis find and how did you add additional features?

Training dataset has 12 columns including target variable "count". Test dataset has 9 feature columns. Feature columns "casual" and "registered" were not present in test dataset so these had to be excluded from training dataset before inputting it for model training.

There are total 10886 records in train dataset and 6493 records in test dataset.

datetime column was not in correct format and had to be converted into datetime64 format. Post conversion into correct format, we extracted date, month and hour as additional features from it in 2nd phase of model training. Season and weather columns were marked as int64 and had to be converted into correct data type since these represent categorical values and not continuous.

How much better did your model perform after adding additional features and why do you think that is?

Following are the reasons, in my opinion, behind better performance of model post addition of additional features:

- 1) Change of season and weather columns to the correct datatype
- 2) Addition of new features – date, month and hour

Model score changed from 1.77 to 0.699 which is a great improvement.

By changing data type to correct format and adding three new features, model was able to extract better fit between dependent and independent variables. For example, when weather is of specific type like hot, there will be less demand of cycles. Same holds true for the hour of the day. And then combining the two variables, if weather is nice, irrespective of the hour of day(except when it is leisure

time/sleeping), demand of cycles will be more. In rainy season, people will be using lesser number of cycles. From hour alone, we can determine when it is night time, there will be very less usage of cycles. Holding everything else constant and assuming good weather and working hours, demand of cycles will be more. This information might not have been evident before we performed feature engineering datetime column.

Hyper parameter tuning

How much better did your model perform after trying different hyper parameters?

Score dropped from 0.699 to 0.49492 which is approx. 29% improvement in performance and quite significant.

If you were given more time with this dataset, where do you think you would spend more time?

That will be on feature engineering and perhaps hyperparameter tuning.

Create a table with the models you ran, the hyperparameters modified, and the kaggle score.

	model	hpo1	hpo2	hpo3	score
0	initial	default	default	default	1.77860
1	add_features	default	default	default	0.69975
2	hpo	{'RF': {'max_depth': Int: lower=4, upper=15, 'max_features': Real: lower=0.6, upper=1.0, 'max_samples': Real: lower=0.7, upper=1.0}}	{'XGB': {'max_depth': Int: lower=4, upper=15, 'subsample': Real: lower=0.6, upper=1.0, 'eta': Real: lower=0.05, upper=0.3, 'lambda': Real: lower=0.7, upper=1.3}}	{'NN': {'num_epochs': 10, 'learning_rate': Real: lower=0.0002, upper=0.01, 'activation': ['relu', 'tanh'], 'dropout_prob': Real: lower=0.02, upper=0.4}}	0.49491

Random Forest:

max_depth: Maximum depth of a tree in random forest

max_features: Number of features to take into account before deciding a split on each node

max_samples: Fraction of total dataset that will be used by a single tree

XGBoost :

max_depth: Maximum depth of a tree in xgboost

subsample: Defines what fraction of total dataset will be used by a single tree

eta: Learning rate

lambda: L2 regularization parameter

Neural Network:

Num_epochs: How many times complete dataset will be passed through training model in entirety

learning_rate – defines the learning rate for model

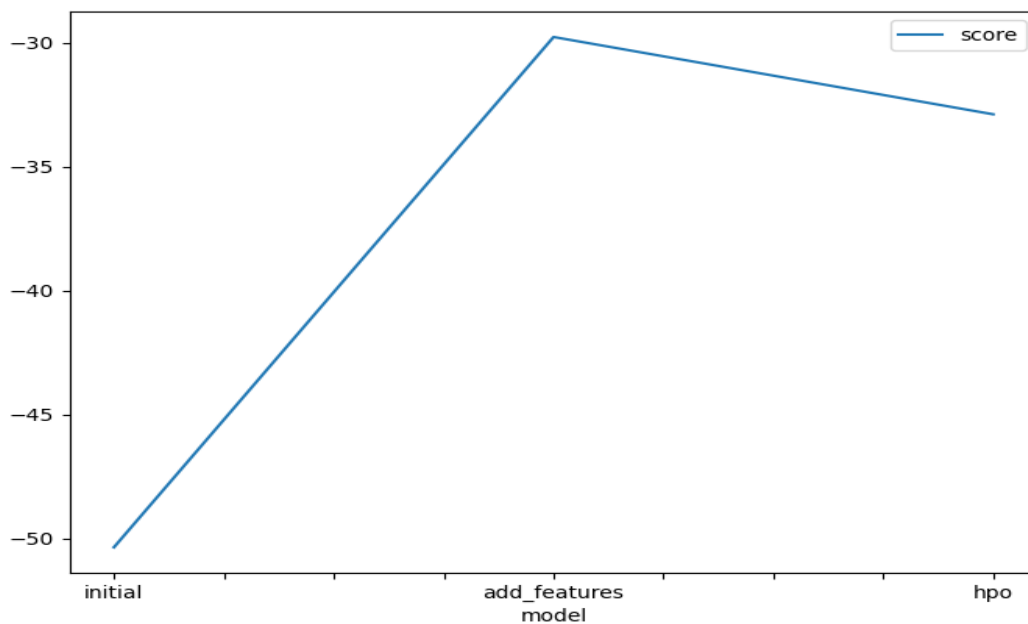
activation – defines the activation function for model

dropout_prob – defines the dropout probability for neurons at a particular node.

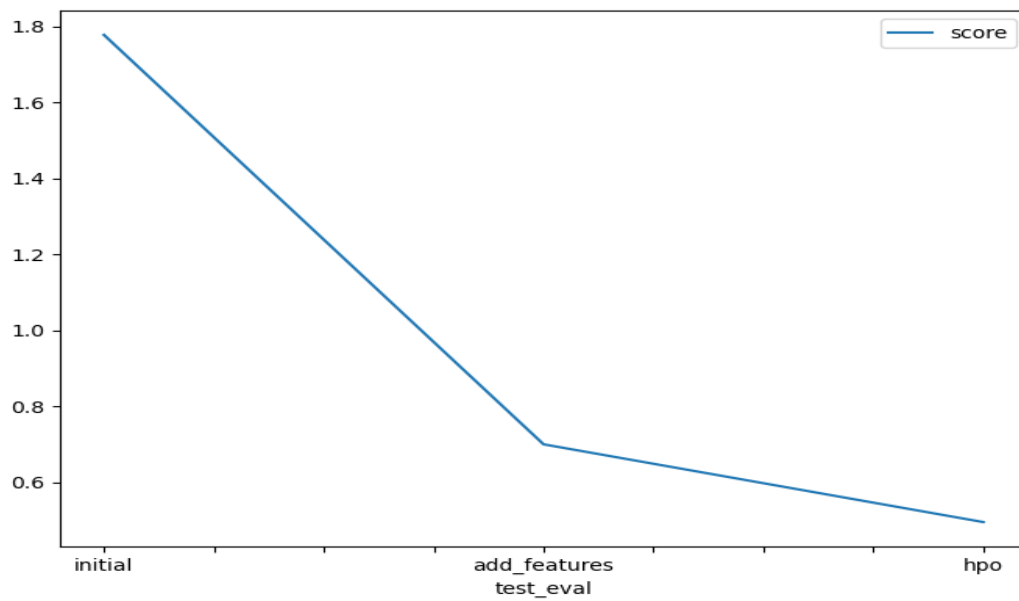
Increasing the tree depth can result in better model fit, however, making it too deep can result in overfitting. Usage of `max_features` and `max_samples/subsample`, `lambda`, `dropout_prob` has a regularization effect on the model and helps in better generalization of the trained model.

Eta/learning_rate is one of the components that determines the speed at which model parameters are adjusted during model training. Gradient descent can be viewed as someone climbing down the hill. Now how fast the person will reach the valley will depend on how steep the gradient is in direction of the movement and how big the step is. Eta/learning_rate corresponds to size of the step.

Create a line plot showing the top model score for the three (or more) training runs during the project.



Create a line plot showing the top kaggle score for the three (or more) prediction submissions during the project.



Summary

Autogluon is a good automl solution and can come as a very useful tool when creating baseline model. In this project, we created models using autogluon with original feature set, then using added features and then performed byperparameter tuning as well with each step resulting in better model performance. Overall a good experience using an automl solution. Would like to explore it more in future.